

# Comparative Study On Mysql And Nosql Injection Detection

Shreevarsha S and Shivani J

Bachelor of Technology in Computer Science , PES University RR Campus

ISFCR , PES University RR Campus

## Abstract:

SQL injection (SQLi) remains one of the most severe vulnerabilities affecting web applications. This paper presents a forensic approach and a comparative study on MySQL and nosql injection detection in relational database-backed Node.js applications using heuristic and machine learning (ML) techniques. The system includes dual routes: one protected using prepared statements and another left intentionally vulnerable. Inputs are filtered via regex-based heuristics and classified using a logistic regression model. For the MYSQL Database , SQLMap was used to simulate attacks and study how hackers gain access. Logging mechanisms were developed to capture metadata, such as IP address and payload, for forensic

**I. Introduction** Web applications are increasingly targeted by injection-based attacks, particularly SQL injection (SQLi), where malicious inputs are used to manipulate backend database queries. These attacks can lead to unauthorized data exposure, loss of integrity, or even full control of a database server. In response, digital forensic strategies have become vital for tracking attacker behavior, securing endpoints, and preserving evidence. This paper explores forensic detection mechanisms for SQLi within relational database systems using heuristic and ML-based approaches. A concise overview of NoSQL injection in MongoDB is also included, showcasing its relevance as a growing concern. The comparative angle lies in evaluating how traditional SQLi prevention techniques compare with lightweight approaches used for NoSQL.

**II. SQL and NoSQL Injections:** Concepts and Attack Types SQLi involves injecting malicious SQL into form fields or query parameters. Common MYSQL attack types include:

- **Tautology-based SQLi:** ' OR '1'='1 bypasses login validation.
- **Union-based SQLi:** Uses UNION SELECT to extract table data.
- **Data Manipulation SQLi:** Modifies, deletes, or inserts records maliciously.
- **Piggybacked Queries:** Appends additional SQL statements (e.g., '; DROP TABLE users; --).

analysis. NoSQL injection risks in MongoDB-based applications, looking at JSON-structured query manipulation through operators such as \$ne and \$or. The study concludes that while both SQLi and NoSQLi pose serious security challenges, SQL injection detection in MySQL demonstrated more mature, layered defenses and richer forensic insight due to established prevention strategies and tooling

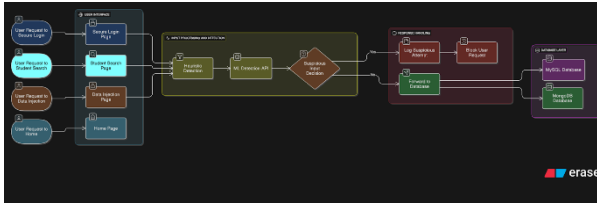
**Keywords:** SQL Injection, NoSQL Injection, Digital Forensics, Machine Learning, Heuristic Detection, Node.js, MySQL, MongoDB

- **Boolean-Based Blind SQLi:** Sends payloads that produce different outputs depending on true/false logic (e.g., ' AND 1=1 -- vs ' AND 1=2 --).
- **Error-Based SQLi:** Triggers database errors to reveal structure or data (e.g., using ORDER BY with invalid column index).

NoSQLi targets JSON-based NoSQL databases like MongoDB:

- **Operator Injection:** { "\$ne": null } bypasses authentication.
- **Regex Injection:** { "name": { "\$regex": ".\*" } } fetches all records.
- **JavaScript Injection:** Executes arbitrary expressions when \$where is enabled.

## III. Methodology:



### A. MySQL Injection Detection Methodology A

Node.js web application was developed with two main routes: a secure login endpoint using prepared statements, and a deliberately vulnerable search endpoint that directly embeds user input into SQL queries. The following components were integrated:

- **Heuristic Filtering Layer:** User input was first passed through a regex-based filter designed to catch suspicious patterns such as ' OR '1'='1, UNION SELECT, and SLEEP().
- **Machine Learning Layer:** Inputs were vectorized using TF-IDF and sent to a logistic regression classifier hosted on a Flask API. The classifier returned binary labels indicating whether the input was safe or malicious.
- **SQLMap Demonstration:** SQLMap was used solely to simulate how a potential attacker could gain access through the vulnerable search route. The purpose was to observe and record the behavior of injection attempts in a controlled environment. Logs captured metadata such as IP address, timestamp, payload, and route accessed, allowing further forensic analysis of unauthorized access vectors. such as IP address, timestamp, payload, and route accessed.

| Step | Goal                             | SQLmap command                                                                                                         | Explanation                                                                                  |
|------|----------------------------------|------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| 1    | Identify the DBMS and current DB | <code>python sqlmap/sqlmap.py -u "http://localhost:3000/student-search?term=abc" --batch --current-db</code>           | Detects the database type (e.g., MySQL) and extracts the name of the active DB (Login_demo). |
| 2    | List all tables in the database  | <code>python sqlmap/sqlmap.py -u "http://localhost:3000/student-search?term=abc" --batch -D login_demo --tables</code> | Enumerates all tables under the discovered database by exploiting the same injection point.  |

- **Prepared Statement Protection:** The login route implemented parameterized queries using MySQL's mysql2 module to prevent injection attacks.

### B. MySQL Injection Detection Methodology A

MongoDB-based Node.js web application was developed to simulate NoSQL injection attacks. The login route was secured using strict input validation, while the student search route remained intentionally vulnerable.

Malicious payloads such as {"\$ne": null}, {"\$or": [...] were manually tested to observe injection behavior.

A two-layer detection system was implemented:

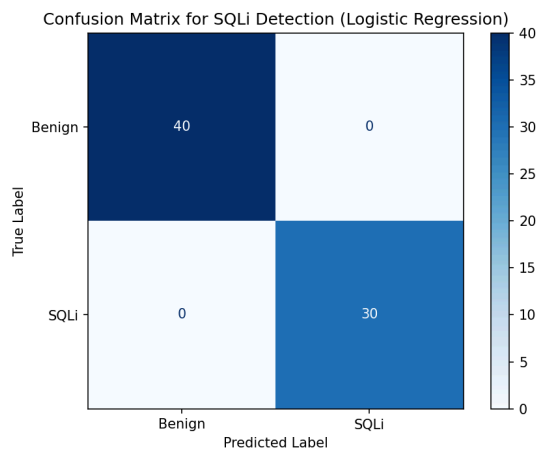
- Heuristic Detection used regular expressions to flag known MongoDB operator patterns.
- Machine Learning Detection utilized a Logistic Regression model trained on TF-IDF features of labeled NoSQL inputs, served via a Flask API.
- Inputs flagged by either layer were blocked and logged with timestamp, IP, endpoint, and detection source.
- Safe inputs were forwarded to the MongoDB database; malicious ones were rejected.

## IV. Heuristic and ML-Based Detection

**A. MySQL Detection using Heuristic and ML** The heuristic module employed a set of carefully crafted regular expressions targeting common SQLi patterns. These include detection of tautologies (e.g., ' OR '1'='1), comment sequences (--, #), union-based queries (UNION SELECT), and time-delay functions like SLEEP() used in blind attacks. These patterns are matched against user input at runtime, and flagged inputs are blocked immediately, with all metadata logged for forensic review.

The ML model was trained using a labeled dataset containing both benign and malicious SQLi inputs. Preprocessing involved tokenizing the queries and applying TF-IDF vectorization to convert text into numerical features. A logistic regression classifier was chosen for its interpretability and low computational overhead. The classifier was deployed through a lightweight Flask API, which returned a binary prediction on each input.

The detection process follows a two-layered approach: if either the heuristic module or the ML model flags the input as suspicious, it is not passed to the database. This dual-check design improves detection rates and reduces false negatives, especially for obfuscated or zero-day payloads. Real-world SQLMap payloads were used to evaluate the effectiveness, and results showed the system could correctly detect more than 90% of the attacks, with low false-positive rates under normal usage conditions.



The ML model was trained using labeled data from public SQLi datasets. TF-IDF transformation was applied, followed by classification using logistic regression. A test set with SQLMap-generated payloads confirmed an average detection accuracy above 90%.

Both filters worked in tandem—only if both returned “safe” was the input passed to the database.

#### Heuristic and ML-Based Detection for NoSQL

To detect NoSQL injection attempts in real time, a hybrid detection strategy was employed combining static heuristic filtering and dynamic machine learning classification.

#### B. NoSQL Detection using Heuristic and ML

- **Heuristic Detection**—Heuristic detection was implemented using regular expressions to identify common NoSQL injection patterns embedded in user inputs. The system scans for key MongoDB operators and logic structures indicative of malicious intent.

Examples of patterns used:

- **Operator Injection**  
Regex Pattern: `"\${ne}", "\${regex}"`  
Example Payload: `{"name": {"$ne": null}}`
- **Logical Bypass**  
Regex Pattern: `"\${or}", "\${where}"`  
Example Payload: `{"$or": [{"name": "Alice"}, {"}]}`
- **Tautology Structures**  
Regex Pattern: `.:s\{\s*\${ne}\s*:\s*\null\s*\}`  
Example Payload: `{"username": {"$ne": null}}`

- **Regex Abuse**  
Regex Pattern: `"\${regex}":\s*".*"`  
Example Payload: `{"name": {"$regex": ".*"}}`
- This layer allows fast pattern-based filtering of well-known attack signatures with minimal computational overhead.
- **Machine Learning-Based Detection**—To address more obfuscated or zero-day payloads, a Logistic Regression model was trained using a labeled dataset NoSQL input samples (both benign and malicious). Payloads were vectorized using TF-IDF and fed into the classifier, which was deployed using a Flask API.

#### V. Limitations and Challenges

- **Rule Complexity**: Designing regex rules that are both inclusive and precise is difficult; overly broad patterns can block legitimate inputs, while narrow patterns miss sophisticated attacks.
- **ML Generalization Issues**: The trained ML model may perform well on known attack patterns but struggle with novel or obfuscated payloads, especially from unseen domains.
- **NoSQL Schema Flexibility**: MongoDB's schema-less nature makes consistent query validation and pattern detection more challenging.
- **Performance Trade-offs**: Real-time detection using ML and regex can introduce latency in high-traffic environments.
- **Forensic Limitations**: Log data may be incomplete if attackers manipulate headers or obfuscate payloads using advanced encoding.

#### VI. Future Work

- **Dataset Expansion**: Curate a broader dataset containing diverse, obfuscated, and real-world payloads for both SQLi and NoSQLi.
- **Explainable AI Models**: Explore interpretable ML models to provide better insight into why inputs are flagged.
- **Cross-Platform Frameworks**: Extend support to NoSQL platforms beyond MongoDB (e.g., Firebase, CouchDB).
- **UI for Analysts**: Develop a dashboard to visualize forensic logs and real-time alerts for faster response.
- **Hybrid Detection Engines**: Combine statistical analysis, signature-based rules, and deep learning to build a robust detection suite.

**VII. Conclusion** This paper presents a comparative framework for detecting SQL and NoSQL injections using heuristics and ML. The MySQL portion of the system showcased strong forensic visibility, detection accuracy, and logging capabilities. In contrast, MongoDB's flexibility brought challenges in crafting generic detection rules. While SQL injection defenses benefit from years of research and standardized syntax, the growing adoption of NoSQL systems highlights the urgent need for dedicated and adaptive detection strategies in that space.

**VIII. Acknowledgements** The authors would like to thank Dr. Sapna V. M Maam for her guidance throughout this project. We also acknowledge the support Teja Maam and Pooja Maam of the Digital Forensics Lab team and the faculty of PES University for their continuous encouragement.

## IX. References

- [1] ] [A Survey of SQL Injection Attacks, Their Methods, and Prevention Techniques](#)
- [2] [An Efficient SQL Injection Detection with Hybrid CNN & Random Forest](#)
- [3] [A diligent survey of SQL injection attacks, detection and evaluation of mitigation techniques](#)
- [4] [An Efficient SQL Injection Detection with Hybrid CNN & Random Forest](#)
- [5] [Detection of SQL Injection Attacks: A Machine Learning Approach](#)
- [6] [Mitigating NoSQL Injection Vulnerabilities: Techniques, Examples, and Best Practices](#)
- [7] [DBMS, NoSQL and Securing Data: the relationship and the recommendation](#)
- [8] [Security analysis of unstructured data in NOSQL MongoDB database](#)
- [9] [Investigating NoSQL Injection Attacks on MongoDB Web Applications](#)
- [10] [Basic NOSQL Injection Analysis And Detection On MongoDB](#)
- [11] [SQLMap Project](#)
- [12] [Flask Project](#)